
RAG-Based Chatbot

Complete Project Documentation

A quick, easy-to-follow guide to the architecture, the vector database, the tools used, and how every part of the pipeline works

AI Training Assignment — Build Your Own RAG-Based Chatbot

September 2026

1. Project Overview

This project is a complete, working Retrieval-Augmented Generation (RAG) chatbot, built for the training assignment: take any document, let a user upload it, and answer questions about it accurately — grounded only in that document's actual content, not the AI's general knowledge.

The assignment asked for the full pipeline to be demonstrated end to end:

Upload	Parse	Chunk	Embed	Store	Retrieve	Generate
--------	-------	-------	-------	-------	----------	----------

What makes this more than a basic demo

- Multiple file types supported: PDF, DOCX, TXT, and Markdown.
- The vector database is persisted to disk — restarting the app does not lose the indexed document.
- Conversation memory — the bot remembers earlier turns in the same session, not just the document.
- Every answer shows its source chunks, so you can verify the bot isn't making things up.
- The bot explicitly refuses to answer from outside knowledge instead of guessing.

2. Architecture & Pipeline Flow

Here is the complete flow of what happens from upload to answer:

1. Upload		2. Parse	
3. Chunk	4. Embed		5. Store
User Question		6. Retrieve	
7. Generate Answer		Show Sources	

- 1 You upload a file (PDF, DOCX, TXT, or MD).
- 2 The app reads the raw text out of it.
- 3 The text is cut into small, overlapping pieces called chunks.
- 4 Each chunk is converted into a list of numbers (an embedding) representing its meaning.
- 5 Those embeddings are saved into a vector database on disk.

- 6 Your question is embedded the same way, and the database finds the closest-matching chunks.
- 7 Those chunks, plus the conversation so far, are sent to Gemini, which writes the grounded answer.

3. Tools & Technologies Used

Everything runs locally in a single Python process — no external servers required.

Tool / Library	Category	Role in this project
Python 3.x	Language	Core programming language for the entire project
Streamlit	Web UI	Chat interface, file upload widget, sidebar controls
LangChain (langchain, langchain-core, langchain-community, langchain-text-splitters)	Orchestration	Document loaders, text splitting, retriever, message types, chaining it all together
langchain-google-genai	Adapter	Lets LangChain call Google Gemini models
Google Gemini gemini-flash-latest	Chat LLM	Generates the final grounded answer from retrieved context + chat history
Google Gemini gemini-embedding-001	Embedding model	Converts text into 3072-number vectors capturing meaning
ChromaDB + langchain-chroma	Vector database	Stores chunk text, embeddings, and metadata; performs similarity search
pypdf	Parsing	Extracts text from uploaded PDF files
docx2txt	Parsing	Extracts text from uploaded DOCX files
python-dotenv	Config	Loads the GOOGLE_API_KEY from the .env file

4. The Database (Vector Store) — Explained in Full

The “database” here is not a traditional SQL table — it's a **vector database**, built to store embeddings (lists of numbers) and quickly find the ones closest in meaning to a query.

4.1 What is ChromaDB?

ChromaDB is an open-source, embedded vector database — it runs inside the same Python process as the app, no separate server needed. It uses SQLite under the hood to persist data to disk, so the index survives app restarts. Chosen here for zero infrastructure, built-in persistence, and tight LangChain integration.

4.2 Where the data physically lives

From `rag_engine.py`:

```
CHROMA_DIR = os.path.join(tempfile.gettempdir(), "rag_chatbot_chroma")
COLLECTION_NAME = "rag_chatbot_docs"
```

On Windows this resolves to a path such as:

```
C:\Users\<you>\AppData\Local\Temp\rag_chatbot_chroma\
```

4.3 What is stored per chunk

Field	What it holds	Example
Document text	The raw chunk text (<code>page_content</code>)	"Full-time employees accrue 18 days of paid vacation..."
Embedding vector	3072 floating-point numbers from <code>gemini-embedding-001</code>	<code>[-0.0108, 0.0332, -0.0018, ...]</code>
Metadata	Where the chunk came from	source: file path; page: page number (PDFs only)

4.4 Collection

All chunks are grouped under one named collection, **`rag_chatbot_docs`** — roughly equivalent to a “table” in a traditional database.

4.5 Persistence & auto-reconnect

Because Chroma writes to disk immediately, the data survives a restart. `app.py` checks on startup whether a populated store already exists and reconnects automatically — so once a document has been processed once, you land straight in the chat on the next run.

4.6 Clearing / resetting

- **Process documents** — wipes the existing collection via Chroma’s own `delete_collection()` API, then rebuilds it from the new file(s).
- **Reset knowledge base** — deletes the entire Chroma folder and clears session state for a completely clean slate.

Note: relying only on deleting the folder from disk (`shutil.rmtree`) can silently fail on Windows when a file handle is still open, stacking old and new chunks together. This was fixed by clearing the collection through Chroma’s API first.

5. Step-by-Step Pipeline Walkthrough

All pipeline logic lives in `rag_engine.py`.

Step 1 — `load_document()`: Parsing

Extension	Loader used
.pdf	PyPDFLoader — reads text page by page
.docx	Docx2txtLoader
.txt / .md	TextLoader (forced to UTF-8 encoding)

Text files are explicitly loaded as UTF-8. Without this, Windows' default encoding turns special characters like an em dash (—) into garbled text such as `â€”` — a mojibake bug caught and fixed during development.

Step 2 — `chunk_documents()`: Chunking

```
chunk_size = 800 characters
chunk_overlap = 100 characters
```

Uses LangChain's `RecursiveCharacterTextSplitter`. Keeping chunks small and focused makes similarity search far more precise; the 100-character overlap prevents ideas from being lost at chunk boundaries.

Steps 3 & 4 — `get_embeddings_model()` + `build_vector_store()`

Each chunk is sent to `gemini-embedding-001`, returning a 3072-number vector. All chunks (vectors + metadata) are written into the Chroma vector database described in Section 4.

Step 5 — `build_retriever()`: Retrieval

Wraps the vector store as a retriever returning the top-k most similar chunks (k adjustable in the sidebar, default 4). The question is embedded the same way the chunks were, then Chroma returns the closest matches.

Step 6 — `answer_question()`: Response Generation

- 1 Retrieve the top-k chunks relevant to the new question.
- 2 Build a system instruction including those chunks as `CONTEXT`, telling the model to answer only from that context (or from earlier chat) — and to admit when it doesn't know.
- 3 Send the system instruction + full conversation history + new question to Gemini.
- 4 Return the answer text plus the source chunks used, so the UI can show exactly where it came from.

Step 7 — `ingest_files()`: Tying it together

Runs Steps 1–4 for every uploaded file in one call — load, chunk, embed, and store — returning the ready-to-use vector store.

6. Conversation Memory

Gemini has no memory between calls — every request is stateless. The app keeps its own running list of the conversation in Streamlit's **session_state.history**, made of HumanMessage and AIMessage objects, and resends the full list on every turn.

This is how the bot can correctly answer “what's my name?” after you introduced yourself earlier — even though your name isn't in the document at all. This memory is session-only: it resets on Clear conversation and does not persist across app restarts (unlike the document knowledge base, which does).

7. Streamlit UI Structure (app.py)

Sidebar

- **Setup** (collapsible) — Gemini API key field; auto-collapses once ready.
- **Knowledge Base** — file uploader, chunks-per-question (k) slider, Process documents button.
- **Status line** — “■ Knowledge base loaded” once everything is ready.
- **Chunk preview** — expandable list of the first indexed chunks.
- **Clear conversation** — resets chat memory only.
- **Reset knowledge base** — wipes the vector database entirely.

Main area

- Pipeline caption restating the seven-stage flow.
- Scrolling chat history.
- Chat input box.
- “Retrieved sources” expander under each answer, showing exactly which chunks (and PDF page) were used.

8. Setup & Run Instructions

One-time setup

- 1 Open a terminal in the project folder: D:\Commas\rag-chatbot
- 2 Create the virtual environment and install dependencies:

```
python -m venv venv
venv\Scripts\pip install -r requirements.txt
```

- 1 Get a free Gemini API key: <https://aistudio.google.com/app/apikey>
- 2 Open the .env file (not .env.example) and set your key:

```
GOOGLE_API_KEY=AIZA...your_real_key...
```

Running the app

```
venv\Scripts\streamlit.exe run app.py
```

Opens the app at <http://localhost:8501>. Upload a document (or use the included sample_docs/acme_handbook.txt), click Process documents, and start chatting.

9. Example Test Prompts

A. Answered correctly from the document

- How many paid vacation days do full-time employees get per year?
- Can I work from home on Mondays?
- What happens if I want to work fully remote?
- How long do I have to submit a receipt for reimbursement?
- What do new employees receive during their first 90 days?
- Is my home internet bill reimbursable?

B. Correctly refused (not in the document)

- What is the capital of France?
- Does Acme Robotics offer health insurance or a 401k match?

Both should get: *"I don't have that information in the uploaded document(s)."*

C. Conversation memory test

- Turn 1: "My name is Alex."
- Turn 2: "What's my name?" — answered from memory, not the document.

10. Troubleshooting Guide

“API key not valid. Please pass a valid API key.”

- The key was typed into `.env.example` instead of `.env` — only `.env` is actually loaded.
- The placeholder text `your_gemini_api_key_here` was never replaced.
- The key was fixed but the app was only refreshed in the browser, not restarted — `load_dotenv()` won't override a value already set in the running process. Stop (Ctrl+C) and re-run streamlit.

“Error calling model ... (NOT_FOUND) — no longer available”

Google periodically retires dated model names. Fixed by switching to **gemini-flash-latest**, an alias Google keeps pointed at their current recommended flash model.

Garbled text like “â€”” instead of an em dash (—)

Caused by reading a UTF-8 file with the wrong default encoding on Windows. Fixed by forcing `TextLoader` to read with `encoding="utf-8"`.

Duplicate / repeated chunks in “Retrieved sources”

Caused by `shutil.rmtree()` silently failing to delete the old Chroma folder on Windows when a file handle was open. Fixed by clearing the collection through Chroma's `delete_collection()` API first.

11. Roadmap / Future Improvements

- 1 Streaming responses — show tokens as they're generated.
- 2 Multi-turn query rewriting — rephrase follow-ups into standalone queries before retrieval.
- 3 Hybrid search — combine keyword search with vector similarity for exact terms (names, IDs).
- 4 Authentication — protect the app if ever deployed somewhere shared.
- 5 Production-grade vector database — swap Chroma for pgvector / Pinecone / Qdrant at scale.
- 6 Evaluation harness — a fixed Q&A test set run automatically after every change.
- 7 Tool-calling / agents — let the model take actions, not just answer questions.

12. Project File Map

```
rag-chatbot/
  app.py           Streamlit UI (upload -> process -> chat)
  rag_engine.py    Core pipeline: load->chunk->embed->store->retrieve->answer
  requirements.txt  Python dependencies
  sample_docs/     Ready-to-use test document (acme_handbook.txt)
  .env             Your real GOOGLE_API_KEY (not committed to version control)
  .env.example     Blank template for the .env file
  README.md        Quick-start guide and build-it-yourself roadmap
  .vscode/settings.json Points VS Code at the project's venv automatically
```

```
venv/
```

```
Local Python virtual environment
```

End of document.